

Certified Optimal Defence in a Path-Contest Network Security Game

September 17, 2026

Abstract

We study a network security game with two players. A defender spreads a fixed budget over the nodes of a directed graph. An evader, after seeing that allocation, picks a route from a source to a destination and spreads its own fixed budget over the nodes on that route. At each node the two investments compete in a ratio-form (Tullock) contest with intensity m . The evader only succeeds if it survives *every* node on its route, so its success probability is a product of per-node survival probabilities.

We prove that, written in logarithmic coordinates, the defender's problem is convex whenever $m \leq 1$, and that it is in fact *strictly* convex, which means the optimal defensive allocation is unique and not just the optimal value. We show by explicit counterexample that convexity fails for $m > 1$. Finding the evader's best route is, in general, an NP-hard problem; we avoid solving it directly by a Lagrangian relaxation that turns "find the worst path among possibly millions" into a single shortest-path computation, together with a rule that proves when no unexamined path can be better. The defender's allocation problem is then solved with a cutting-plane method that produces a genuine two-sided optimality certificate, not just a stopping heuristic.

We validate the method against four independently derived closed-form solutions, matching them to within 5.6×10^{-12} , and pass 33 automated consistency checks. On a graph with 1.68×10^7 possible routes, the solver certifies an answer in under four seconds while examining only two of those routes. We also show that uniform allocation of the budget – reported as empirically good in earlier reliability studies – is exactly optimal on symmetric graphs but 211% worse than optimal on average over 20 irregular graphs, and that a naive "defend the most central node" rule can fail completely.

Keywords: network interdiction; Tullock contest; Stackelberg games; convex optimisation; cutting-plane methods.

Contents

1	Introduction	2
1.1	What this paper contributes	3
1.2	How the paper is organised	3
2	Literature review	3
3	Methodology	4
3.1	Model and notation	4
3.2	The evader's split on a fixed route	5
3.3	Convexity and uniqueness	6
3.4	Finding the evader's best route without checking every route	7
3.5	Solving the defender's problem	7
3.6	Closed-form solutions used for validation	8
3.7	A hand-checkable worked example	8

4	Results	9
4.1	Implementation and testing	9
4.2	Matching the closed forms	9
4.3	Uniqueness check	9
4.4	The worked example	10
4.5	Behaviour across different graph shapes	11
4.6	How much worse the simple rules really are	12
4.7	Does the certificate actually close?	12
4.8	Scaling: why the number of routes is the wrong thing to worry about	12
4.9	Effect of contest intensity	13
4.10	Different graph shapes at a glance	14
4.11	Summary of the computational findings	14
5	Discussion	15
5.1	What the results mean	15
5.2	Where the method stops applying	17
6	Conclusion	17

1 Introduction

A network defender rarely gets to pick where an intrusion happens. The defender chooses only how to spread a limited protective budget across the network, and must commit to that choice before the intrusion occurs. The intruder, by contrast, usually gets to see the defence – guards, checkpoints, and sensors are visible – and routes around the strong points.

This paper studies a direct mathematical version of that situation. A defender (A) spreads a budget $\bar{x}_A$ over the nodes of a directed graph. An evader (B) observes the allocation, picks a route from a source S to a destination D , and spreads a budget $\bar{x}_B$ over the nodes on that route. At each node, the two investments meet in a contest: with defence x_i and attack y_i at node i , the evader passes with probability

$$p_i = \frac{y_i^m}{x_i^m + y_i^m}, \quad (1)$$

where $m > 0$ controls how strongly the larger investment is rewarded. This is the standard ratio-form contest used throughout the conflict-theory literature (Tullock, 1980; Skaperdas, 1996). The evader must survive *every* node on its chosen route, so its overall success probability is the product $\prod_{i \in P} p_i$. The quantity we want to compute is

$$V^* = \min_x \max_P \max_y \prod_{i \in P} \frac{y_i^m}{x_i^m + y_i^m}. \quad (2)$$

At first glance this looks hard to compute exactly. The evader’s choice of route ranges over a set that can be exponentially large; the defender’s allocation ranges over a continuum; and the objective, as written, is neither convex nor concave. Earlier work on closely related models has used either metaheuristics with no optimality guarantee (Ramirez-Marquez et al., 2011), or discrete versions solvable as integer programs (Nguyen et al., 2023). We show that the continuous version above can, for $m \leq 1$, be solved exactly, with a certificate proving the answer is correct.

1.1 What this paper contributes

1. **A convexity result.** Written in log space, the defender’s objective is convex for $m \leq 1$ (Theorem 3.4). This turns (2) into a convex program, which is the reason a certified solution is possible at all.
2. **A uniqueness result.** We strengthen the above to *strict* convexity, which shows the optimal allocation itself is unique, not just the optimal value (Theorem 3.5). Convexity alone does not give this – a convex function can be flat along a whole face – so this needs a separate argument.
3. **A negative result for $m > 1$.** We give a concrete graph on which convexity provably fails at $m = 2$ (Proposition 3.6), so the boundary of the method is not just a proof artefact.
4. **An algorithm that avoids enumerating routes.** Finding the evader’s best route is a maximum-reliability path problem, shown NP-hard by Nguyen et al. (2023). We give a relaxation that reduces this search to one shortest-path computation, plus a rule that certifies when no unexamined route can beat the current best (Section 3.4).
5. **Closed-form checks and a tested implementation.** We derive exact solutions for four graph families and match them numerically to 5.6×10^{-12} (Section 3.6), and report a computational study across nine topologies, 20 random graphs, and graphs with up to 1.68×10^7 routes (Section 4).

1.2 How the paper is organised

Section 2 reviews the closest prior work. Section 3 sets up the model, proves the convexity and uniqueness results, and describes the two algorithms (the evader oracle and the defender’s cutting-plane solver). Section 4 reports the computational study. Section 5 interprets the findings and states the limitations plainly. Section 6 concludes.

2 Literature review

Network interdiction. The classical interdiction literature places an interdictor who changes the network (removing edges, adding delay) against an evader who then routes through what remains (Washburn and Wood, 1995; Israeli and Wood, 2002); see Smith and Song (2020) for a survey. The interaction there is typically sequential, as in our model. Nguyen et al. (2023) study a two-stage game in which an evader crosses a network and its payoff is a product of per-arc survival probabilities – the same series-system structure we use. Their defender’s choices are discrete (which arcs to interdict) and their second stage is simultaneous, which forces mixed strategies and a linear program over path distributions. Our defender’s choice is a continuous budget split and play stays sequential throughout, so a pure (non-mixed) optimum exists. Two ideas carry over directly: their constraint generation over paths is the discrete analogue of our cutting planes (Section 3.5), and their trick of turning a product of survival probabilities into a shortest-path problem via logarithms is the discrete version of our Lagrangian oracle (Section 3.4). Their proof that the underlying path problem is NP-hard is why we do not look for a polynomial exact algorithm.

Contest-based models of protection. Ramirez-Marquez et al. (2011) use the same ratio-form contest $v = T^m / (T^m + t^m)$ and report, as an empirical finding, that splitting the defence budget evenly is optimal when every component is equally vulnerable. We recover this as a proven theorem (Section 3.6) and show exactly where it stops holding: on symmetric graphs it is exact, and it degrades quickly once the graph is not symmetric (Section 4.5). Their solution

method is a genetic algorithm with no guarantee of optimality; our convexity result lets us replace that with a method that proves its own answer, for $m \leq 1$. Hausken (2008) studies attack and defence of series and parallel reliability systems with contest functions, which is the reliability-theory ancestor of the series structure used here; the loss of convexity at high contest intensity noted in that line of work matches our Proposition 3.6.

Attack and interception in networks. The closest model we found in a literature search is Bloch et al. (2023). There, an attacker picks a target and a route, and each node privately decides how much to invest in stopping it. The models differ in every important respect. Bloch et al.’s game is decentralised: each node minimises its own cost, $x_i^2/2$, with no shared budget across nodes, while our defender solves one joint allocation problem under a single budget constraint. Their interception probability is linear in investment, $1 - x_i$, not a ratio-form contest against an opposing investment – indeed their attacker never chooses a continuous effort at all, only a probability distribution over targets. Their solution concept is a simultaneous Nash equilibrium; ours is sequential (Stackelberg). Their uniqueness result is uniqueness of that Nash equilibrium, proved by a fixed-point argument; our uniqueness result (Theorem 3.5) is uniqueness of a jointly-optimised *allocation*, proved from strict convexity. Neither result implies the other, and the two papers answer different questions.

Iliaev et al. (2023) also use a Tullock contest and compare sequential with simultaneous play, but their setting has independent targets and no graph or path structure, so there is no series system and the shortest-path technique that drives our algorithm has no counterpart there. Bozbay and Vesperoni (2018) define a contest success function on a network of friendly and hostile relationships, which is a different use of the word "network" from ours – there is no routing problem in their model at all.

On novelty. This search covered the two papers that motivated the project plus targeted searches on Tullock contests on networks, sequential interdiction with ratio-form contests, and network extensions of the Colonel Blotto game. It is a keyword search, not a systematic citation review, and it should be confirmed with a supervisor or subject expert before any claim of novelty is made in a submission. We have not found an exact match for the combination used here – a single jointly-budgeted defender, a ratio-form contest at each node, a series-system path payoff, and sequential play, solved to a certified optimum – but absence from a search is evidence, not proof.

Algorithms used. The outer solver is Kelley’s cutting-plane method (Kelley, 1960); the gradient it needs comes from Danskin’s theorem (Danskin, 1967); and routes are generated in cost order using Yen’s algorithm (Yen, 1971).

3 Methodology

3.1 Model and notation

Let $G = (V, E)$ be a directed graph with a source S and destination D . Write $\mathcal{P}$ for the set of simple S – D routes. For a route P , write P° for its *contested* nodes – by default every node on P except S and D , since the start and end points of an intrusion are not themselves checkpoints. Let V° be the set of nodes lying on at least one S – D route; budget placed elsewhere affects no payoff, so we restrict attention to V° , with $n = |V^\circ|$.

The defender picks $x \geq 0$ with $\sum_{i \in V^\circ} x_i = \bar{x}_A$. The evader, after seeing x , picks a route P and a split $y \geq 0$ on P° with $\sum_{i \in P^\circ} y_i = \bar{x}_B$. The payoff is

$$U_B(P, x, y) = \prod_{i \in P^\circ} \frac{y_i^m}{x_i^m + y_i^m}. \quad (3)$$

Play is sequential: the defender commits first and the evader responds. This is a Stackelberg game, not a simultaneous one, and only the value V^* in (2) is well defined as a solution concept. A simultaneous version would in general need mixed strategies over routes – this is the case studied by Nguyen et al. (2023) – and we do not treat it here.

Two conventions fix the boundary behaviour of (3) and are not arbitrary choices; each is the only option that keeps the value function continuous.

Zero defence means free passage. If $x_i = 0$, (1) gives $p_i = 1$ for any $y_i > 0$, so the evader spends nothing there. We set $p_i = 1$ when $x_i = 0$; this is the unique extension of the contest function under which the value function stays continuous at the edge of the budget simplex.

Work in logarithms. Define $g(x) = \log U_B = \sum_{i \in P^\circ} [m \log y_i - \log(x_i^m + y_i^m)]$. This avoids numerical underflow on long routes (raw products reach 10^{-300} in our experiments) and, more importantly, this is the coordinate system in which the problem turns out to be convex.

We write $h(x, y)$ for the summand above, $g_P(x) = \max_y h(x, y)$ for the evader’s best split on a fixed route P , and $G(x) = \max_P g_P(x) = \log \max_P \max_y U_B$, so that $V^* = \exp(\min_x G(x))$.

Proposition 3.1 (The defended set must be a cut). *If $\{i : x_i > 0\}$ does not intersect every S – D route, some route is entirely undefended and the evader succeeds with probability 1.*

This is immediate from the free-passage convention: an undefended route gives $p_i = 1$ at every node on it. It is the link to the min-cut view of network interdiction, and it explains one of our experimental results before we get there: a rule that concentrates budget on a few high-centrality nodes can leave an entire route open and so achieve the worst possible outcome.

3.2 The evader’s split on a fixed route

Fix a route P and defence x . For $x_i > 0$, write $\varphi_i(y) = m \log y - \log(x_i^m + y^m)$.

Proposition 3.2 (Strict concavity). *For every $m > 0$ and $x_i > 0$, φ_i is strictly concave on $(0, \infty)$.*

Proof. $\varphi_i'(y) = mx_i^m/[y(x_i^m + y^m)] > 0$, and the denominator is strictly increasing in y while the numerator is constant, so φ_i' is strictly decreasing. $\square$

So the evader’s problem, $\max \sum_i \varphi_i(y_i)$ subject to the budget constraint, has a unique solution $y^*(x)$. Writing λ for the budget multiplier and $t = 1/\lambda$, the stationarity condition is

$$y_i(x_i^m + y_i^m) = t m x_i^m, \quad (4)$$

whose left side is strictly increasing in y_i , giving a unique root $y_i(t)$ found by bisection, with the total spend $\sum_i y_i(t)$ strictly increasing in t . At $m = 1$, (4) is the quadratic $y^2 + x_i y - t x_i = 0$, with cancellation-free positive root

$$y_i(t) = \frac{2x_i t}{x_i + \sqrt{x_i^2 + 4x_i t}}, \quad p_i(t) = \frac{2t}{x_i + 2t + \sqrt{x_i^2 + 4x_i t}}. \quad (5)$$

The textbook form $y = (-x_i + \sqrt{x_i^2 + 4x_i t})/2$ is algebraically the same but should not be used in code: when $4x_i t \ll x_i^2$ it subtracts two nearly equal numbers and loses precision. This matters because the cutting-plane iterates in Section 3.5 approach the boundary of the budget simplex, where exactly this regime occurs.

If every node on the route has the same defence $x_i = c$, symmetry and uniqueness give $y_i = \bar{x}_B/n$ (with $n = |P^\circ|$), so

$$U_B = \left(\frac{\bar{x}_B}{nc + \bar{x}_B} \right)^n. \quad (6)$$

The exponent is the key fact behind almost everything in this paper: the evader’s success shrinks geometrically with route length. A longer route is much harder to survive than a shorter one defended with the same per-node budget, which is why, as we will see, a route with only one checkpoint needs far more defence per node than a route with several.

Proposition 3.3 (Gradient of the inner value). *With y^* the unique optimal split, $\partial g_P / \partial x_i = -m x_i^{m-1} / (x_i^m + (y_i^*)^m)$, which at $m = 1$ is $-1 / (x_i + y_i^*)$.*

This follows from Danskin’s theorem (Danskin, 1967), since g_P is a maximum over a compact set that does not depend on x , and the maximiser is unique. The practical value of this fact is that solving the inner problem once gives both the value and an exact gradient, at no extra cost – this gradient is what drives the outer algorithm in Section 3.5.

3.3 Convexity and uniqueness

Theorem 3.4 (Convexity for $m \leq 1$). *For $0 < m \leq 1$, $G(x) = \max_P \max_y \log U_B$ is convex on the defender’s budget simplex.*

Proof. Fix $y > 0$ and look at one term of $h(x, y)$ as a function of x_i : $-\log(x_i^m + y_i^m)$. Write this as $-\log(u + y_i^m)$ with $u = x_i^m$. For $m \leq 1$, $x_i \mapsto x_i^m$ is concave, and $u \mapsto -\log(u + y_i^m)$ is convex and non-increasing. A convex, non-increasing function applied to a concave function is convex, so each term of $h(\cdot, y)$ is convex in x_i ; the remaining term $m \log y_i$ does not involve x . So $h(\cdot, y)$ is convex in x for every fixed y .

$g_P(x) = \sup_y h(x, y)$ is then a supremum of convex functions taken over a set that does not depend on x , hence convex; and $G = \max_P g_P$ is a maximum of finitely many convex functions, hence convex. The free-passage convention keeps this valid up to the boundary of the simplex. $\square$

This makes (2) a convex program for $m \leq 1$: any local minimum is the global one, and a cutting-plane method returns a real certificate rather than just a number.

Convexity alone gives a unique optimal *value*, but not necessarily a unique optimal *allocation* – a convex function can be flat along an entire edge of the simplex. We close this gap.

Theorem 3.5 (Strict convexity and uniqueness of the optimal defence). *At $m = 1$, G is strictly convex wherever the route that attains the maximum carries a strictly positive evader allocation. The minimiser x^* is therefore unique.*

Proof. Fix a route P and $y > 0$ on P° . Each term $\varphi_i(x_i) = -\log(x_i + y_i)$ has second derivative $1/(x_i + y_i)^2 > 0$ for every $x_i \geq 0$, so it is *strictly* convex in x_i . Because $h(x, y) = \sum_i \varphi_i(x_i)$ sums strictly convex functions of separate coordinates, it is strictly convex in the full vector x : for $x \neq x'$ and $t \in (0, 1)$,

$$h(tx + (1-t)x', y) < th(x, y) + (1-t)h(x', y). \quad (7)$$

Let y^* be the evader’s optimal split at the midpoint $tx + (1-t)x'$. Because $g_P(z) \geq h(z, y^*)$ for any z ,

$$g_P(tx + (1-t)x') = h(tx + (1-t)x', y^*) < th(x, y^*) + (1-t)h(x', y^*) \leq tg_P(x) + (1-t)g_P(x'), \quad (8)$$

using (7) and then $h(\cdot, y^*) \leq g_P(\cdot)$. No equality is needed at x or x' individually – the inequality already points the right way. So g_P is strictly convex. Applying the same device once more, with P^* the maximising route at the midpoint, shows G is strictly convex. A strictly convex function on a compact convex set has exactly one minimiser. $\square$

The argument needs $y_i > 0$ where it uses strict convexity, which fails only when a whole route is undefended – but by Proposition 3.1 that gives $\mathcal{U} = 1$, the worst possible outcome for the defender, so it never happens at the optimum. Uniqueness therefore holds at the actual solution. Section 4 confirms this numerically: starting the solver from three very different initial allocations always converges to the same x^* .

Proposition 3.6 (Convexity fails for $m > 1$). *For $m > 1$, $h(\cdot, y)$ need not be convex, and G is genuinely non-convex on some instances.*

Proof. At $m = 2$, $\frac{d^2}{dx^2}[-\log(x^2 + c)] = \frac{2(x^2 - c)}{(x^2 + c)^2}$, which is negative whenever $x^2 < c$; the argument of Theorem 3.4 then fails. Substituting $u = x^m$ restores convexity of the objective but makes the budget constraint non-convex instead, so no simple change of coordinates fixes this.

To confirm this is a real failure and not just a gap in the proof: on the diamond graph ($S \rightarrow A \rightarrow D$ and $S \rightarrow B \rightarrow D$) with $\bar{x}_A = \bar{x}_B = 10$ and $m = 2$, writing $f(s) = G(s, 10 - s)$, direct computation gives $\frac{1}{2}f(2) + \frac{1}{2}f(4) < f(3)$, with a violation of 2.075×10^{-3} – a direct breach of the convexity inequality. $\square$

For $m > 1$ the solver still runs and the numbers look sensible, but no certificate is available; we label such output NOT PROVEN throughout. This matches the observation in the contest-theory literature that high contest intensity is what pushes such games towards mixed strategies (Hausken, 2008).

3.4 Finding the evader’s best route without checking every route

Computing $G(x) = \max_P g_P(x)$ means solving a maximum-reliability path problem, shown NP-hard by Nguyen et al. (2023). We avoid a brute-force search using Lagrangian duality.

For $\lambda > 0$, define the per-node bound $\psi(x_i, \lambda) = \max_{y \geq 0} [\log p_i(y) - \lambda y] \leq 0$, with equality to 0 exactly when $x_i = 0$. By weak duality, for every $\lambda > 0$ and every route P ,

$$g_P(x) \leq \lambda \bar{x}_B + \sum_{i \in P^\circ} \psi(x_i, \lambda), \quad (9)$$

with equality at the best multiplier λ^* . This bound is *additive over the nodes of P* , so writing $c_i(\lambda) = -\psi(x_i, \lambda) \geq 0$,

$$\max_{P \in \mathcal{P}} g_P(x) \leq \lambda \bar{x}_B - \min_{P \in \mathcal{P}} \sum_{i \in P^\circ} c_i(\lambda). \quad (10)$$

The right side is now an ordinary *shortest-path* problem with non-negative node costs, solved by Dijkstra’s algorithm in time that does not depend on how many routes exist. The bound is tightened over λ by a one-dimensional search.

Fixing the best λ , we then generate routes in increasing cost order (Yen’s algorithm). Since costs only increase along that order, once the k -th route satisfies $\lambda \bar{x}_B - c_\lambda(P_k) \leq$ the best value found so far, no later route can possibly beat the current best, and the search stops with a proof of optimality rather than a guess. This is the mechanism behind the scaling result in Section 4.8: on a graph with 1.68×10^7 possible routes, this stopping rule triggers after examining exactly two of them.

3.5 Solving the defender’s problem

By Theorem 3.4, minimising G is a convex problem, and we solve it with Kelley’s cutting-plane method (Kelley, 1960).

Proposition 3.7 (Cuts are globally valid). *For any route P and any budget-feasible split y (not necessarily optimal), the tangent line to $h(\cdot, y)$ at a current iterate x_k lies below G everywhere on $x \geq 0$.*

This holds because $h(\cdot, y)$ is convex, so it lies above its own tangent, and $h(\cdot, y) \leq g_P \leq G$ by definition. Two things follow. First, the lower bound built from these cuts is robust to a slightly imperfect inner solve: a sub-optimal y still gives a valid, if weaker, cut. Second, because the cut is valid on all of $x \geq 0$, the resulting linear program can be solved without artificially restricting x away from zero, and its optimum is a rigorous lower bound on the true problem.

The algorithm alternates: evaluate the evader's best response at the current allocation (this gives both an exact value and a cut, by Proposition 3.3); solve a small linear program over the accumulated cuts to get a new candidate allocation and a lower bound; track the best value seen so far as an upper bound; and stop when the gap between the two is below a tolerance. Both bounds are rigorous – the upper bound because it is an exact evaluation of G at a real point, and the lower bound by Proposition 3.7 – so the reported gap is a genuine proof of near-optimality, not a claim that the numbers "stopped changing."

Two implementation details matter enough to mention. The lower-bound linear program must allow x down to exactly zero; restricting it to $x \geq x_{\min} > 0$ for numerical convenience shrinks the feasible set and turns the lower bound into an over-estimate, which breaks the certificate. And the upper bound must be evaluated at exactly-sparse points – if the solver is not allowed to actually set some $x_i = 0$, the reported value is biased upward by roughly $\sqrt{x_{\min}/t}$ per node that should be undefined, which is large enough (about 10^{-5} in our runs) to swallow the target tolerance of 10^{-7} .

3.6 Closed-form solutions used for validation

To check the solver against something other than itself, we derive exact answers (all at $m = 1$) for four graph families.

Single chain (n nodes in series): by symmetry and convexity the uniform allocation $x_i = \bar{x}_A/n$ is optimal, giving $V^* = (\bar{x}_B/(\bar{x}_A + \bar{x}_B))^n$ from (6).

k identical parallel chains (each of length n): each branch spreads its share evenly, and since branch value is strictly decreasing in the budget given to that branch, the optimum equalises all branches, giving $a_j = \bar{x}_A/k$ per branch and $V^* = (\bar{x}_B/(\bar{x}_A/k + \bar{x}_B))^n$.

Complete layered graph (L layers of width w , hence w^L distinct routes): symmetry within each layer plus convexity gives a layer-uniform optimum, $c = \bar{x}_A/(Lw)$ per node. Every route meets one node per layer, so it behaves like a chain of length L with per-node budget $Lc = \bar{x}_A/w$, giving

$$V^* = \left(\frac{\bar{x}_B}{\bar{x}_A/w + \bar{x}_B} \right)^L. \quad (11)$$

This is, in our view, the most useful structural fact in the paper: the value depends only on layer width and depth, *not* on the number of routes. A graph with three routes and one with 27 routes can have exactly the same value, and we confirm this numerically in Section 4.5.

Unequal branches (node-disjoint branches of lengths $L_1, \dots, L_r$): every branch must be defended, or it gives $V = 1$ by Proposition 3.1, so all branches are active and equalised at the optimum. Writing V for the common branch value, $a_j = \bar{x}_B(V^{-1/L_j} - 1)$ with $\sum_j (V^{-1/L_j} - 1) = \bar{x}_A/\bar{x}_B$; the left side is strictly decreasing in V , so V is found by bisection.

3.7 A hand-checkable worked example

One graph used throughout the results section has a clean exact solution, useful for sanity-checking by hand. Take $S \rightarrow i_1 \rightarrow i_2 \rightarrow D$, $S \rightarrow j_1 \rightarrow D$, plus a shortcut $i_1 \rightarrow j_1$, with $\bar{x}_A = \bar{x}_B = 10$. There are three routes, with contested sets $\{i_1, i_2\}$, $\{i_1, j_1\}$, and $\{j_1\}$.

Proposition 3.8. *At the optimum, $x_{i_1}^* = x_{i_2}^* = a$ and $x_{j_1}^* = 10 - 2a$, where a solves $a^2 + 15a - 25 = 0$, giving $a = (-15 + \sqrt{325})/2 \approx 1.51387819$, and the game value is exactly*

$$V^* = \frac{7 + \sqrt{13}}{18} \approx 0.5891972930813. \quad (12)$$

Proof. By symmetry between i_1 and i_2 and convexity, the optimum sets $x_{i_1} = x_{i_2} = a$. On the route $\{i_1, i_2\}$ the evader splits evenly, giving value $(5/(a+5))^2$. On the single-node route $\{j_1\}$ the evader spends its whole budget, giving value $5/(10-a)$. Setting these two active routes equal gives $25/(a+5)^2 = 5/(10-a)$, which reduces to $a^2 + 15a - 25 = 0$. Substituting back gives the value in (12). The third route is checked numerically in Section 4.4 to have a strictly lower value, and so is correctly left inactive. $\square$

4 Results

4.1 Implementation and testing

The solver is written in Python, using NumPy, SciPy (for the linear program, via the HiGHS solver) and NetworkX (for shortest paths and route generation). All results below come from a single run of the code on one desktop machine; nothing here is estimated or reconstructed by hand. The source code, the automated test suite, and the scripts used to generate every table and figure in this section are available at [\[repository link\]](#).

The implementation carries 33 automated checks, covering: the inner solver’s optimality against 2.4×10^5 random feasible splits; exact agreement of the analytic gradient with numerical differentiation; convexity along random line segments; the four closed-form families above; agreement between the certified route oracle and brute-force enumeration; agreement with a brute-force grid search on small instances; the vertex-cut necessary condition; behaviour at different contest intensities; and uniqueness of the optimal allocation. All 33 pass.

4.2 Matching the closed forms

Table 1 compares the solver against the four families of Section 3.6, across 20 parameter combinations. The largest error over all 20 instances is 5.6×10^{-12} ; for the chain, parallel, and layered families the error is at the level of floating-point rounding.

Table 1: Solver value versus the closed-form value, for a representative subset of the 20 test instances. “Error” is the absolute difference between the two.

graph family	$\bar{x}_A$	$\bar{x}_B$	solver value	closed-form value	error
chain, 2 nodes	30.0	5.0	0.020408163	0.020408163	3.5×10^{-18}
chain, 4 nodes	10.0	10.0	0.062500000	0.062500000	0
chain, 6 nodes	30.0	5.0	0.000008500	0.000008500	1.0×10^{-20}
2 parallel chains, 2 nodes each	12.0	8.0	0.326530612	0.326530612	5.6×10^{-17}
4 parallel chains, 3 nodes each	12.0	8.0	0.384673178	0.384673178	0
layered, 3 layers x 3 wide (27 routes)	10.0	10.0	0.421875000	0.421875000	5.6×10^{-17}
layered, 3 layers x 4 wide (64 routes)	10.0	10.0	0.512000000	0.512000000	0
unequal branches [1, 2]	10.0	10.0	0.589197293	0.589197293	5.6×10^{-12}
unequal branches [1, 2, 4]	10.0	10.0	0.622422087	0.622422087	4.7×10^{-12}
unequal branches [2, 3, 5]	10.0	10.0	0.440237411	0.440237411	3.5×10^{-12}

4.3 Uniqueness check

The check corresponding to Theorem 3.5 runs the solver from three very different starting allocations – an even split, a random allocation, and one concentrated in a single corner of

the simplex – on four graphs (the worked example, the diamond, the unequal-branches graph, and a 4×4 grid), and compares the resulting x^* across runs. Across all four graphs and all three starting points, the largest coordinate-wise difference observed in the final allocation is 2.0×10^{-5} , at a solver tolerance of 10^{-10} in log-value. In other words, wherever the solver starts from, it lands on the same allocation, which is what Theorem 3.5 predicts.

4.4 The worked example

We solve the graph of Section 3.7 directly. The solver returns $x_{i_1}^* = 1.5138783$, $x_{i_2}^* = 1.5138781$, $x_{j_1}^* = 6.9722436$, and $V^* = 0.5891972930814$, with a certified gap of 1.6×10^{-13} . This matches Proposition 3.8’s exact answer ($a = 1.513878188659973$, $V^* = 0.5891972930813327$) to 6.8×10^{-14} in the value and 7.4×10^{-13} in $x_{j_1}^*$; the two symmetric coordinates $x_{i_1}^*$ and $x_{i_2}^*$ agree with the exact value a to about 1.2×10^{-7} , a looser tolerance because the objective is flat to second order near the minimum, so the value converges faster than the exact split between two interchangeable nodes.

Figure 1 shows the allocation and the evader’s chosen route on this graph. The single-node route through j_1 has no route length to protect it, so it absorbs about 70% of the total budget; the two-node route benefits from the length effect in (6) and needs far less per node.

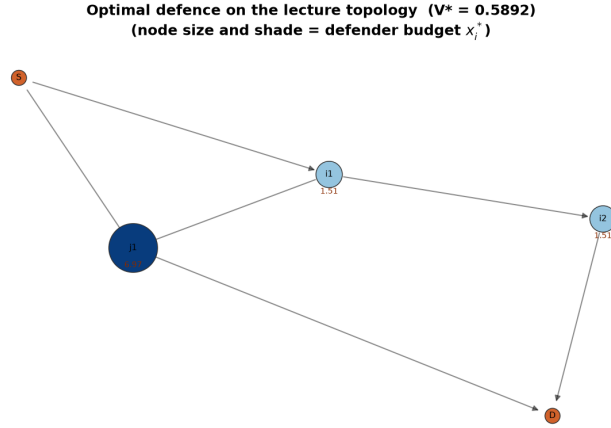


Figure 1: The worked example of Section 3.7. Node size shows the optimal defence budget x_i^* ; j_1 , the only checkpoint on the shortest route, receives almost five times as much as either node on the two-checkpoint route. *Source: generated by the accompanying solver code (`experiments.py`, experiment E2).*

Table 2 compares the certified optimum against five simple rules on this graph. The "most central node" rule places the whole budget on j_1 and leaves the other route completely open, which by Proposition 3.1 gives the worst possible outcome, $V = 1$.

Table 2: Optimal defence versus five simple rules on the worked example ($\bar{x}_A = \bar{x}_B = 10$). "Excess" is how much more often the evader succeeds, relative to the certified optimum.

rule	allocation (i_1, i_2, j_1)	evader success	excess
certified optimum	(1.514, 1.514, 6.972)	0.589	0.00%
greedy (add budget where it helps most right now)	(1.550, 1.500, 6.950)	0.590	0.13%
defend the minimum vertex cut	(0.000, 5.000, 5.000)	0.667	13.15%
defend nodes on the most routes	(4.000, 2.000, 4.000)	0.714	21.23%
spread the budget evenly	(3.333, 3.333, 3.333)	0.750	27.29%
defend the most central node	(0.000, 0.000, 10.000)	1.000	69.72%

4.5 Behaviour across different graph shapes

We solve nine different graphs and check the same five rules on each. Table 3 shows the results. On symmetric graphs – the plain chain, the diamond, three equal parallel chains, a complete layered graph, and a regular grid – spreading the budget evenly is *exactly* optimal, with 0% excess. This is the "equal allocation under equal vulnerability" rule reported empirically by Ramirez-Marquez et al. (2011); here it is a proven theorem (Propositions in Section 3.6), and we can see exactly where it stops working: the moment the graph loses its symmetry (the worked example, unequal branches, a grid with a shortcut added, and a random graph), even splitting evenly can be badly wrong, up to 229% worse than optimal.

Table 3: Certified optimum and excess evader success for five simple rules, on nine graphs ($\bar{x}_A = \bar{x}_B = 10$).

graph	checkpoints	routes	optimum	even split	min. cut	route count	centrality	greedy
chain (4 nodes)	4	1	0.0625	0%	700%	0%	0%	0%
diamond	2	2	0.6667	0%	0%	0%	0%	0%
worked example	3	3	0.5892	27%	13%	21%	70%	0.1%
3 equal parallel chains	9	3	0.4219	0%	78%	0%	0%	0.8%
unequal branches 1/2/4	7	3	0.6224	41%	21%	41%	61%	0.4%
layered, 3x3 (27 routes)	9	27	0.4219	0%	78%	0%	0%	0.8%
grid 4 × 4	16	56	0.4096	0%	95%	27%	0%	0%
grid + short short-cut	18	57	0.5480	48%	52%	80%	83%	0.8%
random graph (12 nodes)	12	13	0.1557	229%	221%	131%	131%	5.7%

Note also that the layered graph (27 routes) and the three equal parallel chains (3 routes) have exactly the same value, 0.4219 – confirming (11)’s claim that the number of routes is not what determines the answer.

Figure 2 illustrates why the even-split rule fails once symmetry is broken: on the regular grid, the optimal allocation *is* the even split, but the moment a short unguarded shortcut is added, or the graph is irregular, the optimum concentrates budget on a handful of structurally important nodes and leaves others nearly undefended, which an even split can never do.

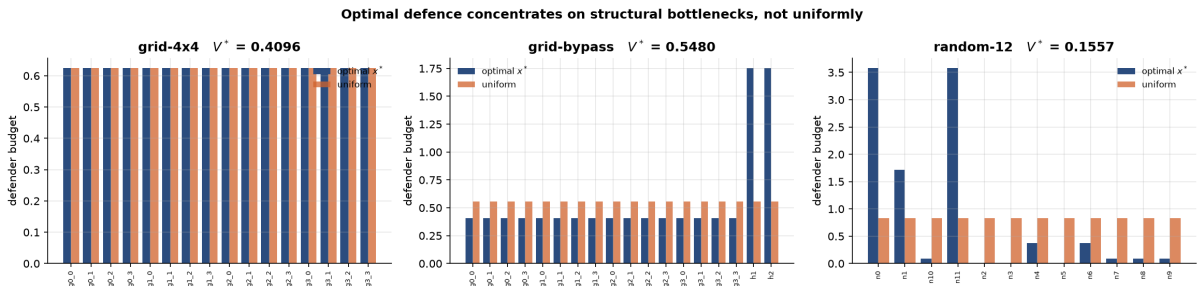


Figure 2: Optimal allocation (dark) against an even split (light) on a regular grid, a grid with a short shortcut added, and an irregular random graph. On the regular grid the two bars are identical; on the other two they diverge sharply, and on the random graph several nodes receive essentially zero defence. *Source: generated by the accompanying solver code (experiments.py, experiment E3).*

4.6 How much worse the simple rules really are

Table 4 and Figure 3 summarise the same five rules across 20 random graphs. Every rule is, as it must be, never better than the certified optimum – but the gap is large. The two rules people reach for first, spreading evenly or defending only a minimum cut, are on average more than three times worse than optimal, and the minimum-cut rule is up to 579% worse in the worst case observed. Only the greedy rule (repeatedly add budget where the current worst-case route is most sensitive to it) comes close, at 10% average excess – and even it reaches 43% in the worst case.

Table 4: Excess evader success over the certified optimum, summarised over 20 random graphs ($\bar{x}_A = \bar{x}_B = 10$).

rule	mean	median	worst case	best case
spread evenly	211%	197%	330%	161%
defend minimum cut only	221%	201%	579%	100%
defend nodes on most routes	125%	123%	167%	91%
defend the most central node	111%	110%	340%	0%
greedy	10%	7%	43%	0%

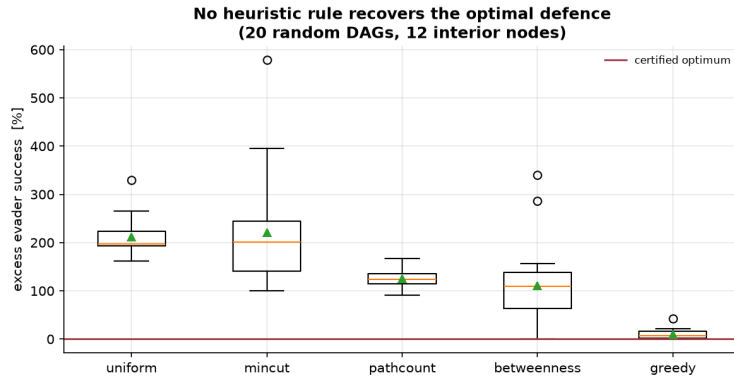


Figure 3: Distribution of excess evader success for five simple rules, over 20 random graphs. The certified optimum is the reference line at 0%; every rule sits well above it. *Source: generated by the accompanying solver code (`experiments.py`, experiment E5).*

The failure of the minimum-cut rule is worth a comment, since Proposition 3.1 might suggest it is a reasonable idea. On the plain 4-node chain it is 700% worse than optimal (Table 3): a chain’s minimum cut is a single node, so the rule puts the entire budget there and throws away the benefit of route length that (6) provides. The optimal defence must touch every route, but it is not true that only a minimum cut should be defended.

4.7 Does the certificate actually close?

On three mid-sized graphs the gap between the upper and lower bound shrinks smoothly to below 10^{-10} , in 43 to 120 iterations (Figure 4). So the certificate is not just a stopping rule – on graphs of this size it genuinely closes to the limits of floating-point precision.

4.8 Scaling: why the number of routes is the wrong thing to worry about

This is the central computational result of the paper. On complete layered graphs, the number of S – D routes grows as w^L , reaching 1.68×10^7 at $L = 12$, $w = 4$. Table 5 and Figure 5 show what happens: solve time grows from 0.14 seconds to 3.70 seconds – a factor of 26 – while the

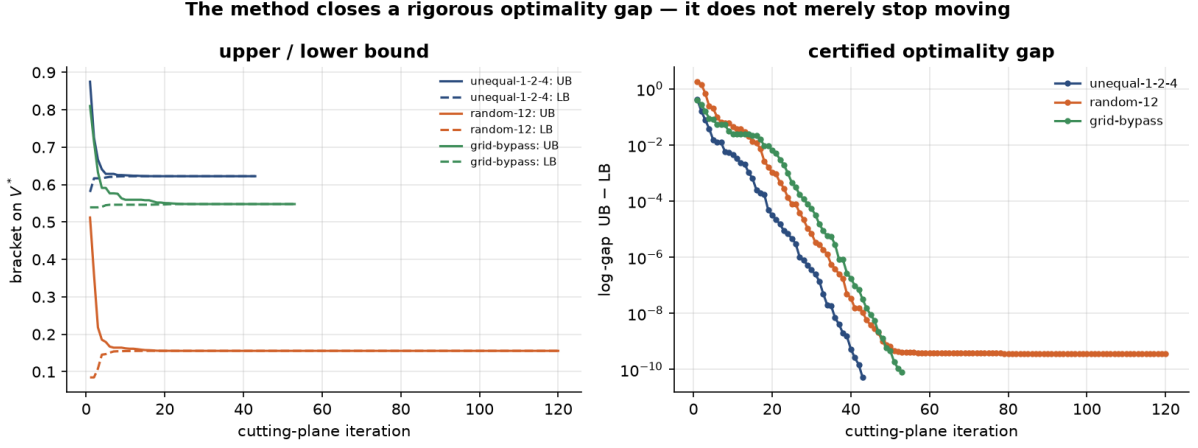


Figure 4: Left: the upper and lower bound on V^* converge to the same value. Right: the gap between them, on a log scale, falls by roughly ten orders of magnitude before flattening out at the limits of floating-point precision. *Source: generated by the accompanying solver code (`experiments.py`, experiment E6).*

number of routes grows by a factor of nearly two million. In every row, the route oracle of Section 3.4 examines exactly two routes before it can prove it has found the best one.

Table 5: Scaling on complete layered graphs ($\bar{x}_A = \bar{x}_B = 10$). “Routes checked” is how many of the listed routes the solver actually examined. Rows at the iteration cap have an exact value but an incomplete proof (see main text).

graph	checkpoints	routes possible	value	routes checked	proof gap	time (s)
2 layers x 3 wide	6	9	0.5625	2	4.4×10^{-10}	0.14
3 layers x 3 wide	9	27	0.4219	2	7.7×10^{-10}	0.29
6 layers x 3 wide	18	729	0.1780	2	9.8×10^{-10}	1.38
8 layers x 3 wide	24	6 561	0.1001	2	1.9×10^{-7}	2.00
10 layers x 3 wide	30	59 049	0.0563	2	2.4×10^{-5}	2.31
8 layers x 4 wide	32	65 536	0.1678	2	5.3×10^{-6}	2.62
10 layers x 4 wide	40	1 048 576	0.1074	2	2.0×10^{-4}	3.16
10 layers x 5 wide	50	9 765 625	0.1615	2	8.8×10^{-4}	4.21
12 layers x 4 wide	48	16 777 216	0.0687	2	2.1×10^{-3}	3.70

What does grow with graph size is the difficulty of *proving* the answer, not the answer itself. The proof-gap column in Table 5 is below 10^{-9} up to 18 checkpoints, but on the largest graphs (48–50 checkpoints) the lower bound has not finished closing within our 200-iteration limit, leaving a gap of up to 2.1×10^{-3} . To be precise about what this means: the *value* in those rows is still exact to floating-point precision (see the closed form comparison), but the *proof* that it is optimal is incomplete. This is a known property of Kelley’s method, which can converge slowly in high dimension (Nesterov, 2018); a more advanced master problem (a proximal-bundle or level-set method) would likely fix this and is a clear next step.

4.9 Effect of contest intensity

Table 6 and Figure 6 vary m from 0.25 to 3. On the plain 4-node chain, the value is exactly 0.0625 for *every* value of m tested. The reason is simple: with equal budgets and only one route, the optimal split is even on both sides, so $x_i = y_i$ at every node regardless of m , and $p_i = \frac{1}{2}$ always. Contest intensity only matters when one side can concentrate its resources and the other cannot.

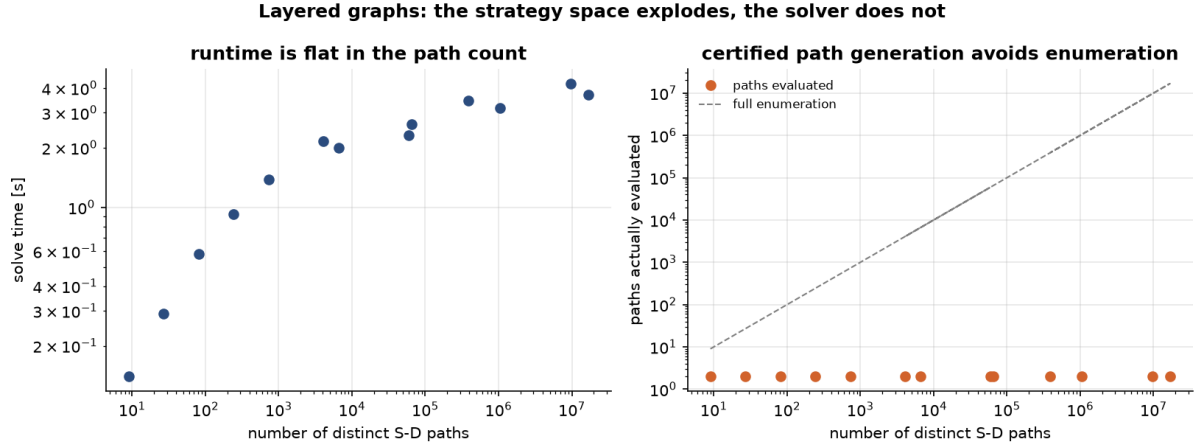


Figure 5: Left: solve time barely grows as the number of possible routes grows across seven orders of magnitude. Right: the number of routes actually examined stays at two, regardless of how many exist, while full enumeration (dashed line) would grow with the graph. *Source: generated by the accompanying solver code (`experiments.py`, experiment E7).*

That is exactly what happens on the grid: the value rises from 0.118 at $m = 0.25$ to 0.940 at $m = 3$. The defender must spread across many parallel routes while the evader can concentrate on one; a higher m rewards concentration, so raising the contest intensity there favours the evader. Rows with $m > 1$ are marked NOT PROVEN, since Proposition 3.6 means no certificate is available there.

Table 6: Certified value as contest intensity m varies ($\bar{x}_A = \bar{x}_B = 10$). Values at $m > 1$ carry no optimality guarantee.

graph	certified				not proven		
	$m = 0.25$	$m = 0.5$	$m = 0.75$	$m = 1$	$m = 1.5$	$m = 2$	$m = 3$
chain (4 nodes)	0.0625	0.0625	0.0625	0.0625	0.0625	0.0625	0.0625
3 parallel chains	0.1835	0.2548	0.3358	0.4219	0.5898	0.7290	0.8966
grid 4×4	0.1177	0.1975	0.2979	0.4096	0.6243	0.7847	0.9399

4.10 Different graph shapes at a glance

Figure 7 shows the optimal defence on nine different graphs side by side, including one with a directed cycle – the method does not require the graph to be acyclic, since only simple (non-repeating) routes matter to the evader. A few patterns are visible directly in the picture. On the unequal-branches graph, the single-node branch gets about 20 times as much defence per node as the four-node branch, for the reason given in Section 3.2. On the random graph, most of the budget concentrates on just two nodes that together form a cut, leaving several nodes at essentially zero – the sparse-support pattern predicted by Proposition 3.1, and here it happens to coincide with the minimum cut, though Section 4.6 shows this coincidence does not hold in general.

4.11 Summary of the computational findings

1. The solver matches four independently derived closed forms to 5.6×10^{-12} , and is never beaten by brute-force search.

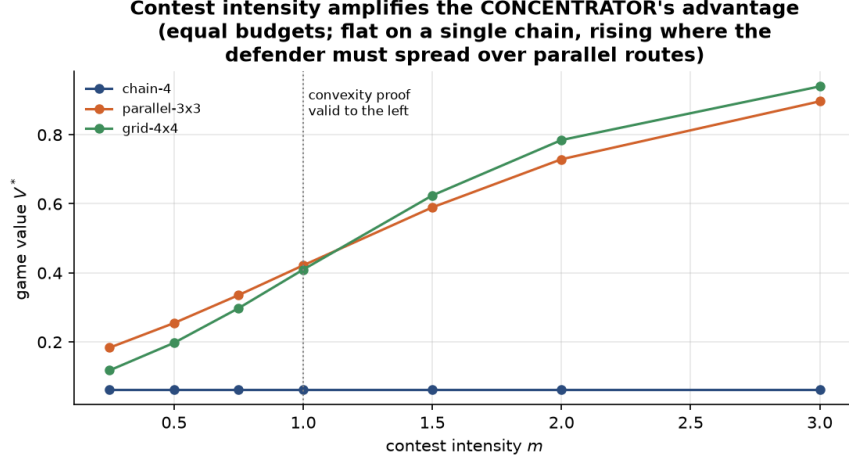


Figure 6: Certified game value against contest intensity m . The vertical line marks the boundary of the convexity proof. The chain is flat because equal budgets on a single route always give a 50/50 contest regardless of m ; the grid and parallel-route graphs rise sharply because the evader can concentrate its effort while the defender cannot. *Source: generated by the accompanying solver code (`experiments.py`, experiment E8).*

2. The number of possible routes is not what makes an instance hard: graphs with 1.68×10^7 routes solve in under four seconds, and graphs with very different route counts can have identical values.
3. Spreading the defence budget evenly is exactly optimal on symmetric graphs and up to 330% worse than optimal on irregular ones.
4. Defending only the most "central" node can fail completely, letting the evader through with certainty.
5. The optimality proof closes to machine precision on graphs up to about 20 checkpoints, but not always within our iteration limit on the largest graphs tested (48–50 checkpoints), even though the numerical answer itself remains accurate.

5 Discussion

5.1 What the results mean

The main practical message is that intuition is not a reliable guide to network defence, even when the intuition sounds reasonable. "Spread the budget evenly" and "protect the choke point" are both defensible-sounding rules, and both can be far from optimal, sometimes catastrophically so. The reason, in both cases, is the same: neither rule accounts for how route *length* interacts with the contest at each node. A short route needs much more defence per node than a long one, because a long route already punishes the evader through repeated multiplication (Equation 6). A rule that ignores this – like spreading the budget by node count, or by how many routes pass through a node – will systematically under-defend short routes and over-defend long ones.

The value of a certified method, rather than a heuristic one, is that it tells you not just an allocation but how far from optimal any other allocation is. Table 4 would not be a meaningful table at all if we could not first solve for the true optimum.

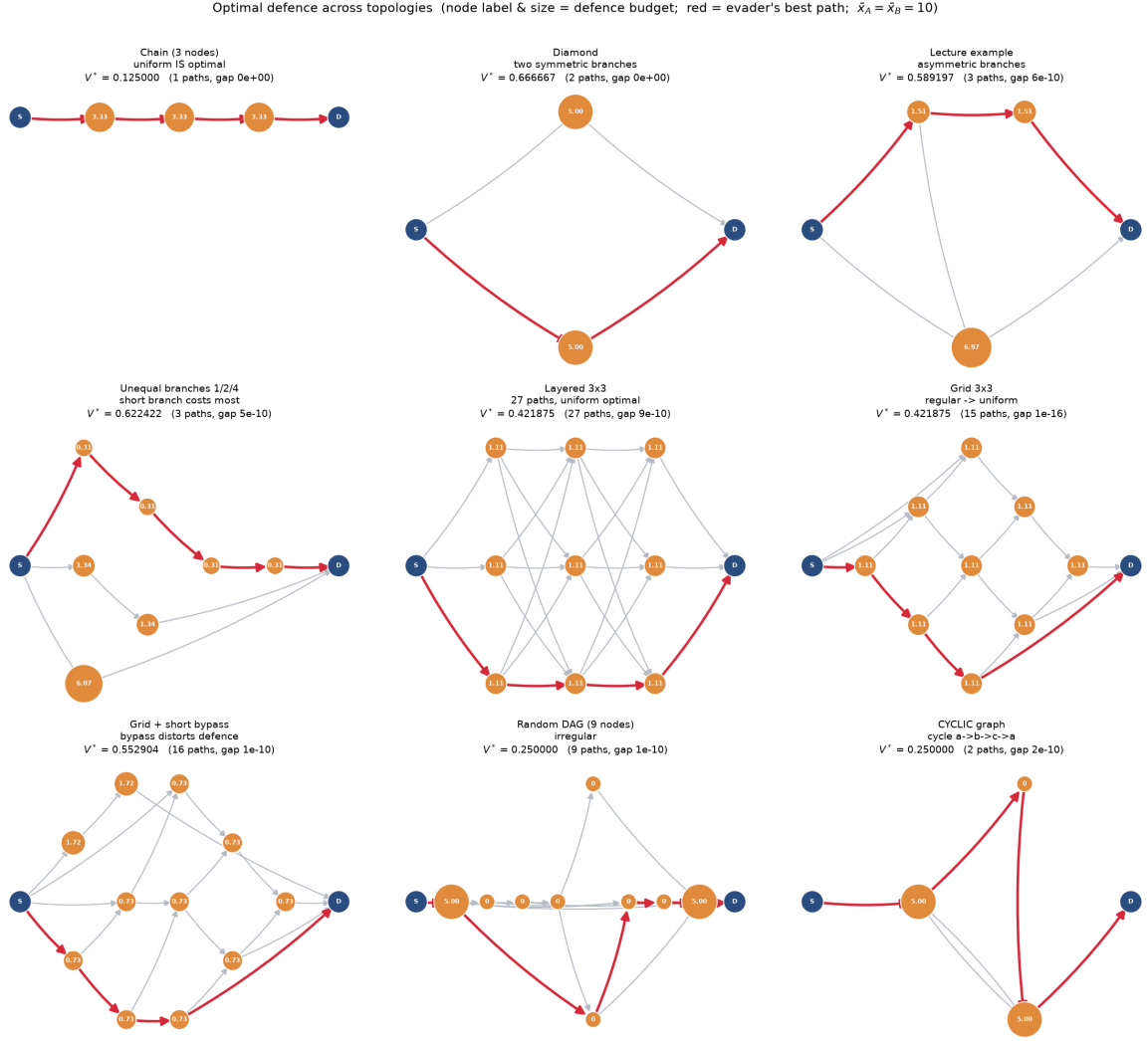


Figure 7: Optimal defence on nine graphs. Node size and label give the defence budget at that node; the red path is the route the evader actually chooses against that defence. The bottom-right graph contains a directed cycle, confirming the method is not restricted to acyclic graphs. *Source: generated by the accompanying solver code ([gallery.py](#)).*

5.2 Where the method stops applying

We list the boundaries of the method plainly, since several of them matter for how the results should be used.

Contest intensity above 1 is not certified.

Convexity provably fails there (Proposition 3.6); the solver still produces numbers, but with no proof attached. Whether any solution concept – pure or mixed – can be characterised in that regime is left open.

The game is sequential, not simultaneous.

The defender commits first. A simultaneous version would generally require mixed strategies over routes, which is the setting studied by [Nguyen et al. \(2023\)](#).

No proven speed of convergence.

We show empirically that the method converges (Section 4.8), but we do not prove how fast. Kelley’s method has no general speed guarantee without extra assumptions on curvature ([Nesterov, 2018](#)); the strict convexity in Theorem 3.5 is a natural starting point for proving one, which we have not done.

The comparison set is generic rules, not competing methods.

We compare against simple, commonly suggested rules – even split, minimum cut, route count, centrality, greedy – not against a re-implementation of the evolutionary method in [Ramirez-Marquez et al. \(2011\)](#) or the integer program in [Nguyen et al. \(2023\)](#) run on the same graphs. A more complete study would include that direct comparison.

Nodes are assumed independent.

The payoff is a product of per-node survival probabilities, which assumes detection events at different checkpoints do not influence each other. Real checkpoints often share information or infrastructure, which would break this assumption.

Only node contests are modelled.

Contests on edges (rather than nodes) would need each edge turned into a node, which is a straightforward change but is not implemented here.

The route oracle is certified but not guaranteed fast.

In the worst case it can still degrade to checking every route, since the underlying problem is NP-hard ([Nguyen et al., 2023](#)).

Uniqueness is proved at $m = 1$.

The proof extends to the interior of the simplex for $m < 1$ using the same argument, but the boundary case (as some $x_i \rightarrow 0$) needs more care and is not written out here.

Novelty has not been independently confirmed.

The literature search behind Section 2 found no exact match, but it is a keyword search, not a systematic review, and should be checked against a supervisor’s knowledge of the area before any claim of novelty is made in a submission.

6 Conclusion

A network defender with a fixed budget, facing an evader who picks both a route and how hard to try at each checkpoint on it, faces a problem that looks intractable at first: an exponential number of possible routes, and an objective that is not obviously convex or concave. We showed that, written in the right coordinates, the problem is convex – and strictly so – whenever the contest intensity does not exceed 1, which gives both a unique optimal value and

a unique optimal allocation. We gave an explicit graph on which this stops being true above that threshold.

Algorithmically, a Lagrangian relaxation that is additive across nodes turns the search over possibly millions of routes into a single shortest-path computation, with a rule that proves when the search can stop; the outer allocation problem then yields to cutting planes with genuine two-sided bounds. In practice, this means graphs with more than sixteen million possible routes solve in a few seconds, examining only two of them.

The applied conclusion is a caution against intuition. Spreading a defence budget evenly, or protecting the most obviously important nodes, are not safe defaults – they are exactly right on symmetric graphs and can be badly wrong the moment a graph is not symmetric, which most real networks are not.

The clearest next steps are: a faster master problem to close the optimality proof on very large graphs; a characterisation of what happens above contest intensity 1; a version of the game where both players move at the same time, which would likely require mixed strategies over routes; and a head-to-head comparison against the solution methods used in the two papers this work is closest to.

Acknowledgements

A note on the use of AI tools

Parts of the software implementation, the literature search, and the drafting of this manuscript were produced with the assistance of a large language model. All mathematical statements were checked by hand by the authors, and every numerical result reported here was produced by the accompanying source code and can be reproduced from it directly.

Data and code availability

All source code, the automated test suite, and the scripts that generate every table and figure in this paper are available at [\[repository link\]](#).

References

- F. Bloch, K. Chatterjee, and B. Dutta. Attack and interception in networks. *Theoretical Economics*, 18(4):1511–1546, 2023.
- I. Bozbay and A. Vesperoni. A contest success function for networks. *Journal of Economic Behavior & Organization*, 150:404–422, 2018.
- J. M. Danskin. *The Theory of Max-Min and its Application to Weapons Allocation Problems*. Springer, Berlin, 1967.
- K. Hausken. Strategic defense and attack for series and parallel reliability systems. *European Journal of Operational Research*, 186(2):856–881, 2008.
- D. Iliaev, S. Oren, and E. Segev. A Tullock-contest-based approach for cyber security investments. *Annals of Operations Research*, 320(1):61–84, 2023.
- E. Israeli and R. K. Wood. Shortest-path network interdiction. *Networks*, 40(2):97–111, 2002.
- J. E. Kelley. The cutting-plane method for solving convex programs. *Journal of the Society for Industrial and Applied Mathematics*, 8(4):703–712, 1960.
- Y. Nesterov. *Lectures on Convex Optimization*. Springer, 2nd edition, 2018.

- D. H. Nguyen, Y. Song, and J. C. Smith. A two-stage network interdiction-monitoring game. *Networks*, 81(3):334–358, 2023.
- J. E. Ramirez-Marquez, C. M. Rocco, and G. Levitin. Optimal network protection against diverse interdictor strategies. *Reliability Engineering & System Safety*, 96(3):374–382, 2011.
- S. Skaperdas. Contest success functions. *Economic Theory*, 7(2):283–290, 1996.
- J. C. Smith and Y. Song. A survey of network interdiction models and algorithms. *European Journal of Operational Research*, 283(3):797–811, 2020.
- G. Tullock. Efficient rent seeking. In J. M. Buchanan, R. D. Tollison, and G. Tullock, editors, *Toward a Theory of the Rent-Seeking Society*, pages 97–112. Texas A&M University Press, 1980.
- A. Washburn and K. Wood. Two-person zero-sum games for network interdiction. *Operations Research*, 43(2):243–251, 1995.
- J. Y. Yen. Finding the k shortest loopless paths in a network. *Management Science*, 17(11):712–716, 1971.